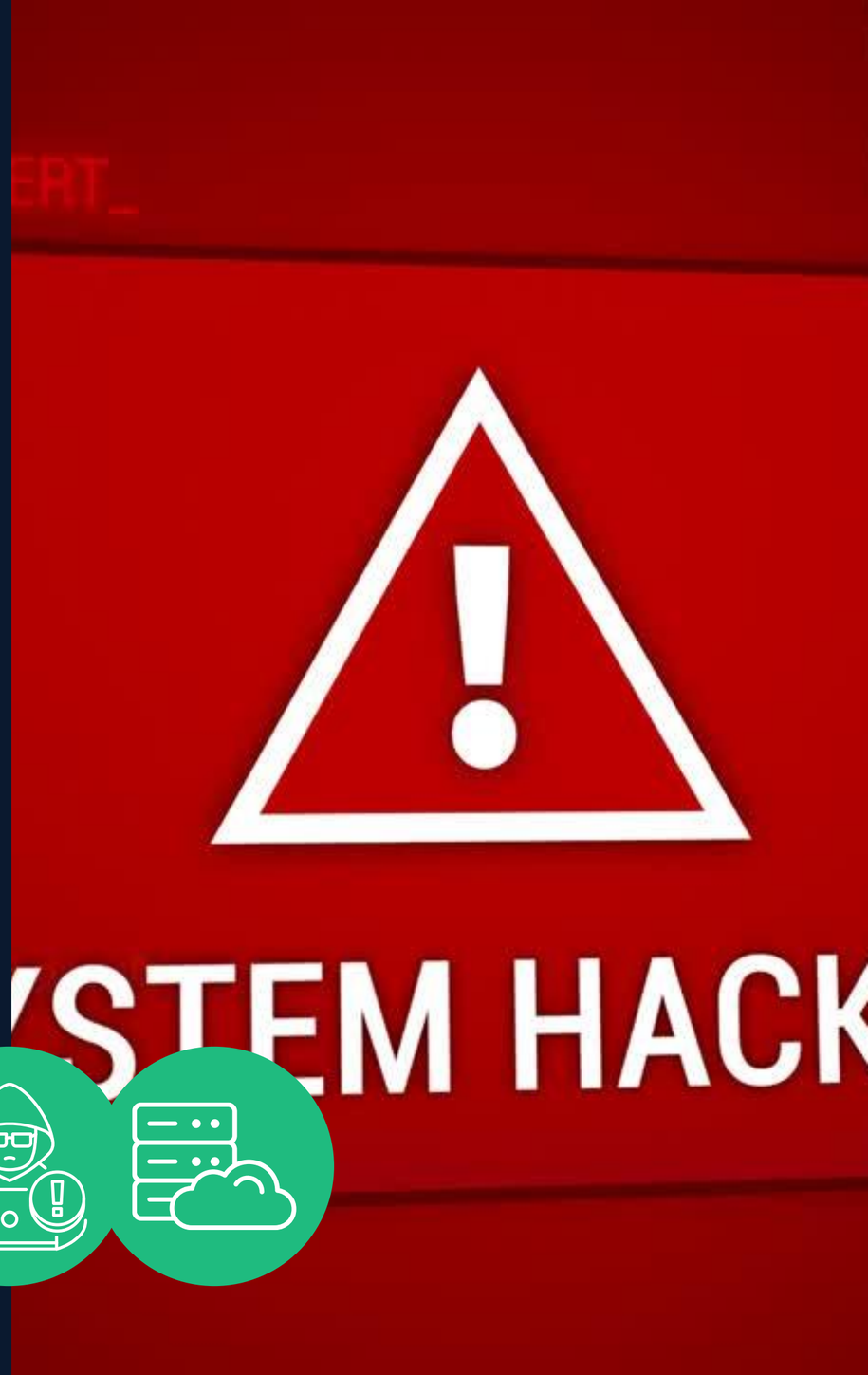
MULTIMEDIA UNIVERSITY, MALAYSIA

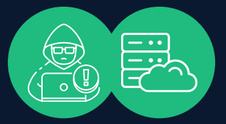
Hacking Web Servers

ETHICAL HACKING AND PENETRATION TESTING
CCS6324 - 2530

Dr. Hafiz Adnan Hussain

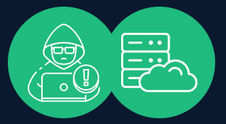


DISCLAIMER



- This course is designed to educate students on ethical hacking principles. The knowledge and techniques acquired must be used responsibly and not for any illegal or unauthorized activities.
- Each student is solely responsible for their actions.
- Do not investigate individuals, websites, servers or conduct any illegal activities on any system you do not have permission to analyze.
- The course content has been developed in good faith, drawing from the author's experience and reputable sources. While every effort has been made to ensure accuracy, the author does not guarantee absolute correctness. Students are encouraged to exercise discretion and actively engage in discussions by raising any questions or concerns during the course.

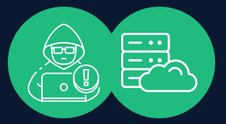
Learning Objectives



What You Will Learn?

- Identify and analyze modern web server architectures and components
- Understand HTTP/HTTPS protocol vulnerabilities and security implications
- Recognize common web server misconfigurations and attack vectors
- Perform web server fingerprinting and vulnerability assessment
- Use industry-standard tools (Nikto, Burp Suite, Gobuster) for security testing
- Understand and mitigate directory traversal and authentication bypass attacks
- Evaluate Web Application Firewall (WAF) capabilities and limitations

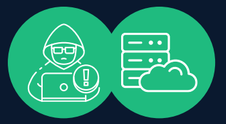
KEY TERMS



Main terminolgies used in the lecture:

- **XSS:** Cross-Site Scripting - injecting malicious scripts into web pages
- **SQL Injection:** Manipulating SQL queries through unsanitized input
- **CSRF:** Cross-Site Request Forgery - tricking users into unintended actions
- **WAF:** Web Application Firewall - traffic filtering and protection
- **TLS/SSL:** Encryption protocols for HTTPS
- **Cookie:** Client-side storage for session state
- **Directory Traversal:** Accessing files outside intended directories
- **Fingerprinting:** Identifying software versions and technologies
- **Reverse Proxy:** Server forwarding requests to backend servers

PART 1: WEB SERVER ARCHITECTURE & COMPONENTS



Modern Web Server Architecture

Web servers today operate in multi-layered architectures rather than simple client-server models:

Simplistic Model:

- HTTP Client (Browser, Mobile App, cURL)
- HTTP Server
- HTTP Request → HTTP Response

Contemporary Enterprise Model:

- Client (Browser/App)
- CDN / Reverse Proxy (CloudFlare, Nginx reverse proxy)
- Load Balancer (distributes traffic)
- Multiple Web Servers (redundancy and scaling)
- Application Layer
- Data Layer (Databases, caches)

PART 1: WEB SERVER ARCHITECTURE & COMPONENTS (Cont'd)

Web Server Technologies

Presentation Layer (HTTP Servers):

- **Apache HTTPD (Unix/Linux, Windows)** - Open-source, modular
- **Nginx** - High-performance, event-driven, increasingly popular for reverse proxies
- **Microsoft IIS** - Windows-native, integrated with Active Directory
- **LiteSpeed** - Commercial alternative, performance-focused

Application Layer:

- **Django (Python)** - Full-featured web framework
- **Rails (Ruby)** - Convention-over-configuration approach
- **Node.js (JavaScript)** - Asynchronous, event-driven
- **Tomcat (Java)** - Enterprise application server
- **ASP.NET Core (Microsoft)** - Modern .NET framework
- **Spring Boot (Java)** - Microservices-oriented

Data Layer:

- **RDBMS:** PostgreSQL, MySQL, MariaDB, Oracle, MSSQL
- **NoSQL:** MongoDB (document), Redis (key-value), Neo4j (graph)
- **Cloud databases:** AWS RDS, Azure SQL Database, Google Cloud SQL

PART 1: WEB SERVER ARCHITECTURE & COMPONENTS (Cont'd)

Web Application Components

Static Web Pages:

- Pure HTML content, no server-side processing
- **Vulnerability:** Directory traversal, information disclosure

Dynamic Web Pages:

- Form processing (<form> tags, input validation)
- Server-side scripting (PHP, ASP.NET, Python, Node.js)
- Client-side scripting (JavaScript, TypeScript)
- Database connectivity (SQL queries, ORM frameworks)
- Session management (cookies, tokens, JWT)

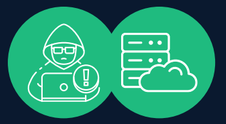
Common Gateway Interface (CGI):

- Legacy technology for executing external programs
- Rarely used in modern applications, but may exist in legacy systems
- Security concerns: insecure file uploads, command injection

Modern Alternatives to CGI:

- FastCGI (persistent processes)
- Application servers (Tomcat, Node.js, Django)
- Serverless functions (AWS Lambda, Google Cloud Functions)

PART 2: HTTP/HTTPS PROTOCOL FUNDAMENTALS



What is HTTP/HTTPS?

HTTP (HyperText Transfer Protocol) is the basic set of rules for web communication, sending data in plain text between browsers and servers, while HTTPS (HTTP Secure) is its secure version that uses SSL/TLS encryption to scramble sensitive data like passwords and credit card numbers, protecting it from interception and ensuring privacy with a padlock icon in browsers. The key difference is encryption: HTTP is insecure (plaintext), HTTPS encrypts data for secure transfer, requiring an SSL/TLS certificate for verification and secure handshakes.

HTTP Versions

- **HTTP/0.9:** Minimal, GET-only, no headers
- **HTTP/1.0:** Basic headers, each request opens a new connection
- **HTTP/1.1 (most common):** Persistent connections, pipelining, improved caching
- **HTTP/2:** Binary framing, multiplexing, header compression
- **HTTP/3:** QUIC protocol, faster connection establishment, UDP-based

PART 2: HTTP/HTTPS PROTOCOL FUNDAMENTALS (Cont'd)

HTTP Request Structure

POST /api/login HTTP/1.1
Host: example.com:8080
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 31
Connection: keep-alive
Cookie: sessionid=abc123

username=admin&password=secret

Components:

- **Method:** GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH, TRACE, CONNECT
- **URI:** Path and parameters (/path/file.php?key=value)
- **Protocol Version:** HTTP/1.1, HTTP/2, HTTP/3
- **Headers:** Metadata about the request
- **Body:** Optional data (POST, PUT, PATCH)

HTTP Response Structure

HTTP/1.1 200 OK
Server: Apache/2.4.52
Date: Mon, 22 Dec 2025 10:15:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 1234
Set-Cookie: sessionid=xyz789; HttpOnly; Secure; SameSite=Strict
Cache-Control: no-cache, no-store, must-revalidate

<html>
<body>Login successful</body>
</html>

Components:

- **Status Line:** HTTP version, status code, reason phrase
- **Response Headers:** Metadata, cookies, security directives
- **Body:** Requested resource or error message

PART 2: HTTP/HTTPS PROTOCOL FUNDAMENTALS (Cont'd)

HTTP Status Codes

Code	Category	Examples
1xx	Informational	100 Continue, 101 Switching Protocols
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

PART 2: HTTP/HTTPS PROTOCOL FUNDAMENTALS (Cont'd)

URL Encoding & Parameters

http://example.com:8080/path/file.htm?key1=val1&key2=val2#anchor

Structure:

- **Scheme:** http, https, ftp
- **Domain:** example.com
- **Port:** 8080 (default 80 for HTTP, 443 for HTTPS)
- **Path:** /path/file.htm
- **Query Parameters:** ?key1=val1&key2=val2
- **Fragment:** #anchor

URL Encoding (special characters must be encoded):

- ? → %3F
- = → %3D
- & → %26
- # → %23
- Space → %20 or +
- / → %2F

Session Cookies

HTTP is stateless; cookies maintain session state:

Set-Cookie: sessionid=abc123xyz; Domain=.example.com; Path=/;
HttpOnly; Secure; SameSite=Strict; Max-Age=3600

Security Attributes:

- **HttpOnly:** Prevents JavaScript access (mitigates XSS cookie theft)
- **Secure:** Only transmitted over HTTPS
- **SameSite:** Strict (no cross-site), Lax (same-site + top-level navigations), None (cross-site)
- **Max-Age/Expires:** Session duration

PART 2: HTTP/HTTPS PROTOCOL FUNDAMENTALS (Cont'd)

HTTPS & TLS/SSL

HTTPS Advantages:

- Encryption (protects data in transit)
- Authentication (verifies server identity via certificates)
- Integrity (detects tampering)

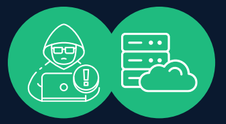
Modern TLS Versions:

- **TLS 1.2:** Still widely supported, acceptable, but aging
- **TLS 1.3:** Latest, faster, more secure, requires modern infrastructure
- **SSL 3.0, TLS 1.0, 1.1:** DEPRECATED, should be disabled

Certificate Validation Issues:

- Self-signed certificates (testing environments)
- Expired certificates (configuration errors)
- Mismatched domain names
- Certificate chain issues

PART 3: WEB SERVER VULNERABILITIES



Information Disclosure Through Headers

Web servers often leak sensitive information in HTTP response headers:
Server: Apache/2.4.52
X-Powered-By: PHP/8.1.0
X-AspNet-Version: 4.0.30319
X-Frame-Options: SAMEORIGIN

Attack Vector: Attackers use this information to search for known vulnerabilities.

Mitigation:

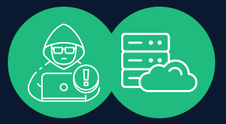
- Remove or obfuscate server identification headers
- Use reverse proxies to hide backend architecture
- Implement security headers (CSP, X-Frame-Options, etc.)

Default Credentials
Many web servers and management interfaces ship with default credentials:

Software	Default User	Default Password
IIS	Administrator	(varies)
Tomcat	tomcat	tomcat
Apache Axis2	admin	axis2
Jenkins	admin	admin
WordPress	admin	(user-set)

Attack Vector: Brute-force or dictionary attacks against login interfaces.

PART 3: WEB SERVER VULNERABILITIES (Cont'd)



Misconfigurations

Directory Listing:

- Unprotected directories returning file listings
- Allows attackers to discover sensitive files

Incorrect Permissions:

- World-readable configuration files containing credentials
- Writable directories allowing file uploads

Outdated Software:

- Running vulnerable versions of Apache, IIS, or frameworks
- Missing security patches

Debug Mode Enabled:

- Detailed error messages revealing source code
- Stack traces exposing internal paths

Directory Traversal (Path Traversal)

Attackers use relative path sequences to access files outside intended directories:

../../../etc/passwd

..\..\..\..\windows\system32\config\sam

%2e%2e%2fconfig.php (URL encoded)

Vulnerable Code Example:

```
$file = $_GET['page'];
include("pages/" . $file . ".php");
```

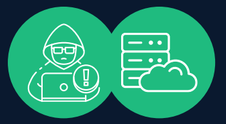
Attack:

http://example.com/view.php?page=../../../etc/passwd

Mitigation:

- Whitelist allowed files/directories
- Use basenames to prevent path traversal
- Chroot/jail environments
- Proper file permissions

PART 3: WEB SERVER VULNERABILITIES (Cont'd)



SQL Injection

Attackers manipulate SQL queries through unsanitized input:

Vulnerable Code:

```
$name = $_GET['username'];
$query = "SELECT * FROM users WHERE username='$name'";
```

Attack:

http://example.com/search.php?username=' OR '1'='1

Resulting Query:

```
SELECT * FROM users WHERE username='' OR '1'='1'
```

It always returns true, bypassing authentication.

Modern SQL Injection:

- Union-based injection (extracting additional data)
- Blind SQL injection (inferring results from response differences)
- Time-based blind (using delays to exfiltrate data)
- Second-order injection (stored in database, executed later)

Mitigation:

- Parameterized queries/prepared statements
- Input validation and sanitization
- Principle of least privilege (database user permissions)
- Web Application Firewalls (WAF)

Cross-Site Scripting (XSS)

Attackers inject malicious JavaScript into web pages:

Reflected XSS:

http://example.com/search.php?q=<script>alert(document.cookie)</script>

Stored XSS:

- Malicious payload stored in the database
- Executed for all users viewing the content
- **Example:** Comments, forum posts, user profiles

DOM-based XSS:

- JavaScript modifies the page without a server round-trip
- **Vulnerable code:** document.body.innerHTML = userInput

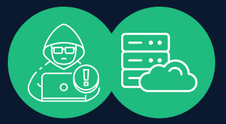
Consequences:

- Steal session cookies
- Keylogging
- Credential harvesting
- Defacement
- Drive-by downloads

Mitigation:

- Content Security Policy (CSP)
- HTML entity encoding
- Input validation
- HttpOnly cookies

PART 3: WEB SERVER VULNERABILITIES (Cont'd)



Cross-Site Request Forgery (CSRF)

Attacker tricks an authenticated user into performing an unintended action:

Attack Scenario:

1. User logs into the banking website
2. User visits attacker's website (in another tab)
3. Attacker's site contains: <img src="bank.com/transfer?amount=1000&to=attacker">
4. The browser automatically includes authentication cookies
5. Unauthorized transfer occurs

Mitigation:

- CSRF tokens (unique per request/session)
- SameSite cookie attribute
- Custom headers (X-Requested-With)
- Referer validation

Broken Authentication & Session Management

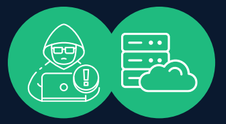
Issues:

- Weak passwords
- Session fixation (attacker sets known session ID)
- Session timeout is too long
- Insufficient logout (sessions not invalidated)
- Exposed session tokens in logs/URLs

Consequences:

- Account takeover
- Privilege escalation
- Unauthorized data access

PART 3: WEB SERVER VULNERABILITIES (Cont'd)



Insecure File Uploads

Vulnerabilities:

- No file type validation
- Uploading executable files (.php, .asp, .jsp)
- Predictable upload locations
- Path traversal in a filename

Attack:

Upload file: shell.php.jpg
Server saves to: /uploads/shell.php (misconfigured)
Attacker executes: http://example.com/uploads/shell.php

Mitigation:

- Whitelist allowed file types
- Store uploads outside web root
- Rename files with random names
- Disable script execution in the upload directory
- Validate file content (magic bytes, not just extension)

Web Server-Specific Vulnerabilities

Apache:

- Mod_ssl vulnerabilities
- .htaccess misconfigurations
- Default error pages exposing version info

IIS/Windows:

- NTLM relay attacks
- WebDAV vulnerabilities
- Windows authentication bypass

Nginx:

- Path traversal in configuration
- Incorrect MIME type handling
- Reverse proxy bypass techniques

PART 4: WEB SERVER FINGERPRINTING & RECONNAISSANCE

Passive Fingerprinting

HTTP Headers Analysis:
curl -I https://example.com
Reveals: Server version, framework, technologies used
Tools:

- Netcraft (web-based, historical data)
- WhatWeb (command-line, multiple techniques)
- Wappalyzer (browser extension)

Active Fingerprinting

HTTP OPTIONS Request:
curl -X OPTIONS -v https://example.com

May reveal allowed HTTP methods:
Allow: GET, POST, HEAD, OPTIONS, PUT, DELETE

Dangers:

- WebDAV methods enabled (PUT/DELETE)
- TRACE method enabled (cross-site tracing, XST attack)

Banner Grabbing

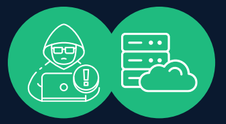
Direct Connection:
nc example.com 80
telnet example.com 80

Type HEAD / HTTP/1.1 and press Enter twice.

Web Server Identification Tools:

- Nikto (automated fingerprinting + scanning)
- Shodan (search engine for internet-connected devices)
- SSL Certificate analysis (SSL Labs)

PART 5: WEB APPLICATION FIREWALL (WAF)



WAF Purpose & Operation

Functions:

- Protects against OWASP Top 10 vulnerabilities
- Monitors and filters HTTP traffic
- Blocks known malicious patterns
- Rate limiting and DDoS protection

Deployment:

- Cloud-based (Cloudflare, AWS WAF, Akamai)
- On-premise (ModSecurity, F5 ASM)
- Reverse proxy integrated (Nginx, Apache)

WAF Detection Techniques

WAF Presence Indicators:

- Unusual error messages ("Your request was blocked")
- Delayed responses
- Specific header patterns (X-Powered-By WAF)
- Consistent 403/406 responses to payloads

WAF Bypass Techniques

1. Encoding Variations:

Original: <script>alert(1)</script>

Double URL: %253Cscript%253E

HTML entities: <script>

Unicode: \u003cscript\u003e

2. Protocol Manipulation:

- HTTP/2 smuggling
- Content-Type switching
- Multipart encoding

3. Logic Bypass:

- Case variation: ScRiPt, SCRIPT
- Null bytes: <scri\x00pt>
- HTML comments: <script>/comment/alert(1)</script>
- JSON payloads instead of form data

4. Timing & Volume:

- Slow requests (evade detection patterns)
- Distributed requests across IPs/time

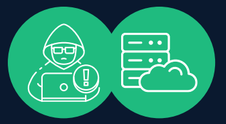
5. Evasion via Headers:

- X-Forwarded-For spoofing
- Custom headers with payloads

Legal & Ethical Considerations:

- WAF bypass only during authorized penetration tests
- Document the testing scope in the rules of engagement
- Use isolated lab environments for technique development

PART 6: WEB ATTACK TOOLS



Network Reconnaissance Tools

Whois & DNS:
whois example.com
nslookup example.com
dig example.com
Reveals: Registrant info, DNS records, email servers

Nikto - Web Server Scanner
Features:
- Identifies web server types and versions
- Scans for known vulnerabilities
- Checks for dangerous files/misconfigurations
- SSL/TLS analysis

Usage:
nikto -h example.com
nikto -h example.com -p 8080
nikto -h example.com -Tuning x (skip executable checks)
nikto -h example.com -o report.html

Burp Suite - Web Application Testing Platform

Core Modules:
1. Proxy:
- Intercept HTTP/HTTPS traffic
- Modify requests/responses
- Analyze communication

2. Scanner:
- Passive scanning (analyzes traffic)
- Active scanning (sends payloads)
- Detects: XSS, SQL injection, CSRF, misconfigurations

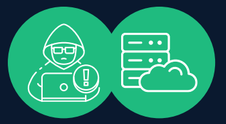
3. Intruder:
- Automated payload injection
- Brute-force attacks
- Parameter fuzzing

4. Repeater:
- Manual request modification
- Testing specific payloads

5. Sequencer:
- Session token randomness analysis
- Cryptographic strength assessment

Free vs. Pro:
- Free: Proxy, manual testing, limited scanning
- Pro: Full automation, advanced features, support

PART 6: WEB ATTACK TOOLS (Cont'd)



Directory Brute-forcing Tools

Gobuster:

gobuster dir -u http://example.com -w /usr/share/wordlists/dirb/common.txt
gobuster dir -u http://example.com -w wordlist.txt -x php,html,txt
gobuster vhost -u http://example.com -w subdomains.txt

DirBuster:

- GUI-based directory/file discovery
- Large wordlists included
- Slower than Gobuster but more user-friendly

FFuF (Fuzz Faster U Fool):

ffuf -u http://example.com/FUZZ -w wordlist.txt
ffuf -u http://example.com/admin.php?user=FUZZ -w usernames.txt

WhatWeb - Web Technology Fingerprinting

whatweb example.com
whatweb --aggressive example.com
whatweb --log-verbose=report.txt example.com

Identifies: CMS, plugins, server software, JavaScript libraries

Metasploit Framework (Web Modules)

Relevant Modules:

auxiliary/scanner/http/dir_scanner
auxiliary/scanner/http/dir_listing
auxiliary/scanner/http/http_version
exploit/windows/iis/iis_webdav_scsf_bypass
exploit/unix/webapp/php_include

Usage:

msfconsole
use auxiliary/scanner/http/dir_scanner
set RHOSTS example.com
run

PART 7: DEFENSE & MITIGATION STRATEGIES

Secure Configuration

Web Server Hardening:

- Disable unnecessary modules (CGI, WebDAV, TRACE method)
- Remove default files (test pages, examples)
- Update to the latest patched version
- Run with minimal privileges (non-root user)

Example (Apache):

```
# Remove TRACE method
TraceEnable Off

# Disable directory listing
<Directory /var/www/html>
   Options -Indexes
</Directory>

# Hide server version
ServerTokens Prod
ServerSignature Off
```

Example (Nginx):

```
server_tokens off;
client_max_body_size 10M;

location ~ /\. {
   deny all;
   access_log off;
   log_not_found off;
}
```

Security Headers

HSTS (HTTP Strict Transport Security):

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

Content Security Policy (CSP):

Content-Security-Policy: default-src 'self'; script-src 'self' trusted.cdn.com; style-src 'self' 'unsafe-inline'

X-Frame-Options (Clickjacking):

X-Frame-Options: DENY

X-Content-Type-Options:

X-Content-Type-Options: nosniff

Web Application Firewall (WAF) Deployment

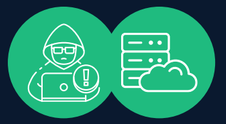
Cloud WAF (Cloudflare, AWS WAF):

- Managed rule sets (OWASP, vendor-specific)
- Geo-blocking
- Bot management
- Easy to deploy (DNS/proxy change)

ModSecurity (Open-source):

```
# Enable ModSecurity
SecEngineOn
# Load OWASP CRS rules
Include /usr/share/modsecurity-crs/*.conf
# Custom rules
SecRule ARGS:id "^[0-9]+$" "phase:2,deny,status:403"
```

PART 7: DEFENSE & MITIGATION STRATEGIES (Cont'd)



Input Validation & Output Encoding

Server-side (PHP):

```
// Prepared statements (prevent SQL injection)
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?");
$stmt->execute([$user_id]);

// Input validation
$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
$age = filter_input(INPUT_POST, 'age', FILTER_VALIDATE_INT);

// Output encoding (prevent XSS)
echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');
```

Client-side (Supplementary only, not primary defense):

```
// Input validation
const email = document.getElementById('email').value;
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
  alert('Invalid email');
}
```

Logging & Monitoring

Web Server Access Logs:

192.168.1.100 - - [22/Dec/2025:10:30:45 +0000] "GET /admin.php HTTP/1.1" 200 1234 "-" "Mozilla/5.0"

Look for:

- Multiple 404 responses (scanning activity)
- SQL injection patterns in parameters
- Unusual User-Agent strings
- Directory traversal attempts

Security Information & Event Management (SIEM):

- ELK Stack (Elasticsearch, Logstash, Kibana)
- Splunk
- Azure Sentinel

Summary

Modern web server security requires understanding multi-layered architecture, HTTP protocol mechanics, and diverse attack vectors. Organizations must implement defense-in-depth strategies combining secure configuration, input validation, WAF deployment, and continuous monitoring. Security professionals must master both attack techniques and mitigation strategies to protect critical web infrastructure.

ADDITIONAL RESOURCES

- OWASP Top 10: https://owasp.org/Top10/
- OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
- CWE Top 25: https://cwe.mitre.org/top25/
- PortSwigger Web Security Academy: https://portswigger.net/web-security
- HackTricks Web Penetration Testing: https://book.hacktricks.xyz/
- Burp Suite Documentation: https://portswigger.net/burp/documentation

Thank You

ANY QUESTIONS?